

Pop-Culture & Video-Game Examples

Arrays, 2D/3D, and ArrayLists

Same concepts as the traditional set, dressed up for the nerds in the room. Organized by topic so you can grab a 2-D example or an `ArrayList` example on demand; each is tagged with its franchise so you can read the room and pick. All compile as written.

A note on the shy nerds. Sprinkling these in signals “this is a safe place to know this stuff,” and the quiet Trek/Who kids tend to light up and start answering. You do not have to get every canon detail right; half the fun is a student correcting you, which is exactly the participation you want. Data below is invented or common knowledge, no need to defend it.

A. One-dimensional review

A1. A Pokémon team of six

[Pokémon / video games]

An array is perfect here: a team is *exactly* six slots, no more.

```
String[] team = {"Pikachu", "Charizard", "Snorlax", "Gengar", "Lapras", "Alakazam"};
System.out.println("Party lead: " + team[0]);
team[5] = "Dragonite";           // swap the last slot
for (int i = 0; i < team.length; i++) {
    System.out.println((i + 1) + ". " + team[i]);
}
```

Ask the class: why is a fixed array right for a party of six, but wrong for a Pokédex? (Hold that thought for section F.)

A2. The Doctors, by number

[Doctor Who]

```
String[] doctors = {"Hartnell", "Troughton", "Pertwee", "T. Baker", "Davison"};
// The "Nth Doctor" is a natural fit for a zero-vs-one indexing talk:
int n = 4;                               // the Fourth Doctor...
System.out.println("The 4th Doctor: " + doctors[n - 1]); // ...lives at index 3
```

Ask the class: the “Fourth Doctor” is at index 3; this off-by-one is the whole reason we harp on zero-based indexing. Where else does “the Nth thing” bite you?

A3. Shared reference: the transporter clone bug

[Star Trek]

```
String[] awayTeam = {"Kirk", "Spock", "McCoy"};
String[] pattern = awayTeam;           // NOT a copy -- same buffer
pattern[2] = "Redshirt";               // "beam up a redshirt instead"
System.out.println(java.util.Arrays.toString(awayTeam)); // Predict...
```

Predict: who is in `awayTeam` now? (The transporter did not make a copy; both names point to one pattern buffer. Beam through a `copyOf` if you want two distinct away teams.)

B. Two-dimensional arrays

B1. Battleship / DRADIS grid

[Battlestar Galactica / classic games]

```
char[][] dradis = new char[8][8];
for (char[] row : dradis) java.util.Arrays.fill(row, '.'); // empty space
dradis[3][5] = 'B';   // a Basestar contact at row 3, col 5
dradis[6][1] = 'R';   // a Raider
for (char[] row : dradis) {
    System.out.println(new String(row));
}
System.out.println("Contact at [3][5]: " + dradis[3][5]);
```

Ask the class: is `[row][col]` or `[x][y]` the right mental model for a tactical grid? (Either works; the point is to *commit* to one. Great moment to make them defend a choice.)

B2. Pokémon type-effectiveness chart

[Pokémon / video games]

A 2-D array of multipliers is *exactly* how games store this.

```
// index order: 0 = Fire, 1 = Water, 2 = Grass
double[][] effectiveness = {
    // vs Fire  vs Water  vs Grass
    { 1.0, 0.5, 2.0 }, // Fire attacking
    { 2.0, 1.0, 0.5 }, // Water attacking
    { 0.5, 2.0, 1.0 }  // Grass attacking
};
int attacker = 1; // Water
int defender = 0; // Fire
System.out.println("Damage multiplier: " + effectiveness[attacker][defender] + "x"); //
2.0x
```

Ask the class: read `effectiveness[2][1]` out loud, Grass attacking Water, and predict the number before we index it. Then: how would you find each type's single best matchup? (A max-across-the-row loop.)

B3. A platformer tilemap

[video games]

```
// 0 = sky, 1 = ground, 2 = coin
int[][] level = {
    {0, 0, 2, 0, 0},
    {0, 0, 0, 0, 0},
    {1, 1, 1, 1, 1}
};
int coins = 0;
for (int[] row : level) {
    for (int tile : row) {
        if (tile == 2) coins++;
    }
}
System.out.println("Coins in this level: " + coins);
```

Extend live: add a rule “print P where the player spawns (first sky tile of the top row).” Two minutes, and it forces them to reason about row 0 specifically.

B4. Rogue Squadron roster grid

[Star Wars]

```
// rows = flights, cols = positions in the flight
String[][] squadron = {
    {"Luke", "Wedge", "Biggs"},
    {"Wes", "Hobbie", "Janson"}
};
for (int flight = 0; flight < squadron.length; flight++) {
    System.out.print("Flight " + (flight + 1) + ": ");
    System.out.println(String.join(" ", squadron[flight]));
}
```

Ask the class: `squadron.length` versus `squadron[0].length`: which is “number of flights” and which is “pilots per flight”? Force the answer before moving on.

C. Ragged arrays

C1. Companions per Doctor

[Doctor Who]

Different Doctors travelled with different numbers of companions: a textbook ragged array.

```
String[][] companions = new String[3][];
companions[0] = new String[]{"Susan", "Ian", "Barbara"}; // 3
companions[1] = new String[]{"Jamie", "Zoe"}; // 2
companions[2] = new String[]{"Sarah Jane", "Harry", "Leela", "K9"}; // 4
for (int d = 0; d < companions.length; d++) {
    System.out.println("Doctor " + (d + 1) + " travelled with "
```

```

        + companions[d].length + ": "
        + String.join(", ", companions[d]));
    }

```

Ask the class: why would a *rectangular* 2-D array be the wrong tool here? (You would waste slots and invent fake companions to pad the short rows.)

D. Three-dimensional arrays

D1. A Minecraft-style voxel chunk

[Minecraft / video games]

The canonical honest use of a 3-D array: a cube of blocks.

```

// [x][y][z], each cell a block id: 0 = air, 1 = stone, 2 = grass
int[][][] chunk = new int[4][4][4];
for (int x = 0; x < 4; x++) {
    for (int z = 0; z < 4; z++) {
        chunk[x][0][z] = 1;    // floor layer: stone
        chunk[x][1][z] = 2;    // next layer up: grass
    }
}
System.out.println("Block at (2, 0, 3): " + chunk[2][0][3]);    // 1 = stone

```

Ask the class: name the three axes in your own words. When is a real cube of data worth three brackets, and when are you just showing off?

D2. A dungeon with multiple floors

[roguelikes / video games]

```

// [floor][row][col] -> '#' wall, '.' floor, '>' stairs down
char[][][] dungeon = new char[2][3][3];
for (char[] floor : dungeon)
    for (char[] row : floor)
        java.util.Arrays.fill(row, '.');
dungeon[0][1][1] = '>';    // stairs down on floor 0
dungeon[1][0][0] = '#';    // a wall on floor 1
System.out.println("Floor 0, center tile: " + dungeon[0][1][1]);

```

Ask the class: the outer index is “which floor.” What real thing does *each* index mean? Getting them to narrate the axes is the entire 3-D lesson.

E. The Arrays utility class

E1. Sort the Enterprise crew, then look someone up

[Star Trek]

```
String[] crew = {"Spock", "Kirk", "Uhura", "McCoy", "Sulu", "Scott"};
java.util.Arrays.sort(crew); // alphabetical, in place
System.out.println(java.util.Arrays.toString(crew));
int pos = java.util.Arrays.binarySearch(crew, "Kirk"); // valid ONLY after sorting
System.out.println("Kirk is at index " + pos);
```

Ask the class: try the search *before* sorting and watch it return nonsense with no error. Why is a wrong answer with no crash worse than a crash?

E2. Are these two fleets identical?

[Star Wars]

```
String[] fleetA = {"X-wing", "Y-wing", "A-wing"};
String[] fleetB = {"X-wing", "Y-wing", "A-wing"};
System.out.println(fleetA == fleetB); // Predict
System.out.println(java.util.Arrays.equals(fleetA, fleetB)); // Predict
```

Predict both. (`==` asks “same array object?”; `Arrays.equals` asks “same ships in the same order?”.)

F. ArrayList

F1. An RPG inventory that grows and shrinks

[video games]

The reason `ArrayList` exists: you loot, you sell, the count changes constantly.

```
import java.util.ArrayList;

ArrayList<String> inventory = new ArrayList<>();
inventory.add("Potion");
inventory.add("Iron Sword");
inventory.add("Shield");
inventory.add("Potion"); // duplicates are fine
System.out.println("Carrying " + inventory.size() + " items: " + inventory);

inventory.remove("Shield"); // sold it
System.out.println("Potions in bag? " + inventory.contains("Potion"));
System.out.println("First potion slot: " + inventory.indexOf("Potion"));
```

Ask the class: why is an array a bad fit for an inventory but a great fit for equipped gear (head/chest/legs/boots)? (Fixed slots versus an open-ended pile.)

F2. The Doctor’s ever-growing list of companions

[Doctor Who]

```
import java.util.ArrayList;
```

```

ArrayList<String> companions = new ArrayList<>();
companions.add("Rose");
companions.add("Jack");
companions.add("Martha");
companions.add("Donna");
companions.remove("Jack");           // he wanders off
System.out.println(companions.size() + " aboard the TARDIS: " + companions);

```

Ask the class: across the whole show you never know the final companion count; why is that *exactly* the situation an `ArrayList` is built for?

F3. The `remove()` trap: a Pokédex mix-up

[Pokémon / video games]

```

import java.util.ArrayList;

ArrayList<Integer> pokedex = new ArrayList<>();
for (int id = 1; id <= 6; id++) pokedex.add(id); // [1, 2, 3, 4, 5, 6]
pokedex.remove(2);                             // Predict: index or value?
pokedex.remove(Integer.valueOf(5));             // Predict: index or value?
System.out.println(pokedex);

```

Predict each line. (`remove(2)` deletes the entry at *index* 2 → the value 3. `Integer.valueOf(5)` deletes the *value* 5.) The most valuable 90 seconds in the unit: the game-flavored numbers make the index-versus-value confusion land hard.

F4. Purge the destroyed ships: safe removal while iterating

[Battlestar Galactica]

```

import java.util.ArrayList;
import java.util.Iterator;

ArrayList<String> fleet = new ArrayList<>(
    java.util.List.of("Galactica", "Pegasus", "WRECK", "Colonial One", "WRECK"));
Iterator<String> it = fleet.iterator();
while (it.hasNext()) {
    if (it.next().equals("WRECK")) {
        it.remove();           // safe; enhanced-for + fleet.remove would throw
    }
}
System.out.println("Survivors: " + fleet);
// One-liner: fleet.removeIf(s -> s.equals("WRECK"));

```

Ask the class: rewrite it as `for (String ship : fleet) fleet.remove(ship);`, run it, and watch the `ConcurrentModificationException`. Seeing the fleet crash is more memorable than a warning about it.

F5. Which structure? A rapid-fire round

[mixed]

Fire these off and have them shout “array” or “list”:

- A chessboard → **array** (2-D, fixed 8×8)
- Your RPG inventory → **list** (grows/shrinks)
- A Pokémon party of six → **array** (fixed size)
- The Pokédex you fill in as you play → **list** (open-ended)
- The pixels of a sprite → **array** (2-D grid)
- The Doctor’s companions over the whole show → **list** (unknown count)
- A tic-tac-toe board → **array** (fixed 3×3)
- Ships still in the fleet after a battle → **list** (count keeps dropping)

The takeaway to land: *fixed grid of primitives* → *array*; *growing pile of things* → *list*. When it is a toss-up, start with the list for readability and switch to an array only if you have a measured reason to.